

Acrobot

Merrick Summers and Jonathan Wu

ENME440

UMD

Spring 2026

<https://github.com/aMerrick/440-acrobot-project>

Problem Definition

The goal of the acrobot problem is to swing the free end of a two-link chain up past a target height in the smallest number of steps possible, by controlling the torque on the joint between the links.

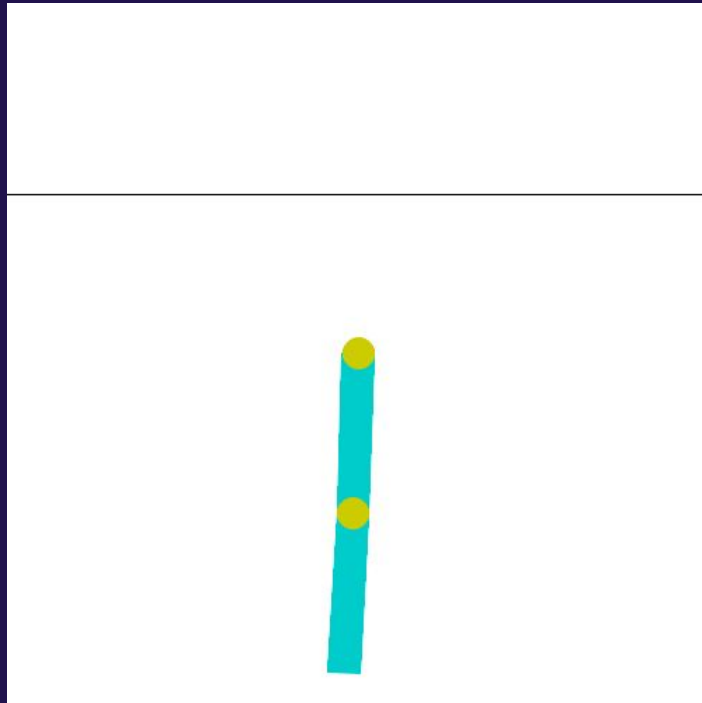
At each step we:

- observe the state of the system,
- choose an action, and
- receive a penalty if the episode is not finished

The loop is:

$s \rightarrow a \rightarrow s' \rightarrow a' \rightarrow \dots$ (plus a penalty for each non-terminal step)

The loop ends if the free end of the acrobot reaches the target height.



Problem Definition - Environment

6 dimensions:

$$\begin{matrix} \sin(\theta_1) \sin(\theta_2) \\ \cos(\theta_1) \cos(\theta_2) \\ \omega_1 \quad \omega_2 \end{matrix}$$

Actions:

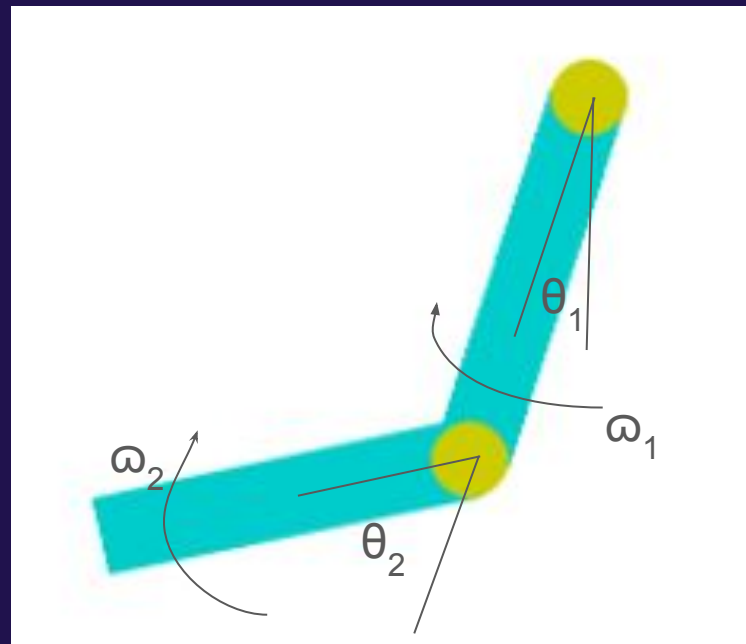
- -1 torque
- 0 torque
- +1 torque

Reward:

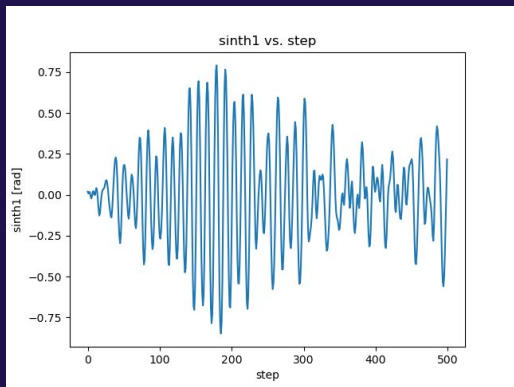
- -1 per timestep until the free end reaches the target height
- Episode ends immediately on success

Success Metric:

- Average reward over 100 consecutive episodes
 - A value > -100 indicates a valid problem solution



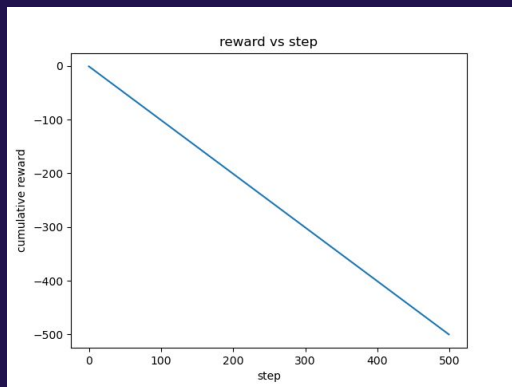
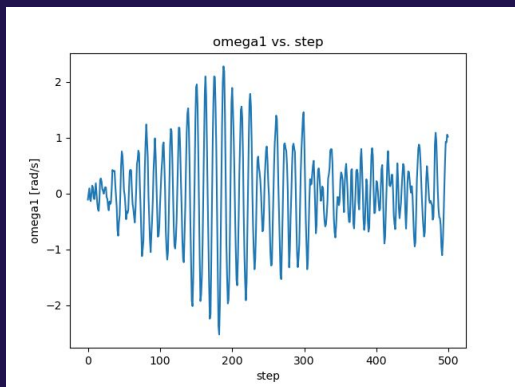
Initial Visualizations



Our first visualizations were of two state space variables and the reward received by a randomly acting model. These visualizations helped us to realize that:

- Rewards are sparse
- The data is smooth
- Observations are not sparse
- Oscillations are present

These visualizations helped us to form a basic understanding of the problem parameters, but were not useful in characterizing the performance of models.



Data Pre-Processing

For the baseline model, observations were sorted into discrete states based on a predetermined set of bin ranges.

This was the only form of data pre-processing used in this project, as the observations generated through interaction with the environment were already in a good format, and did not need operations like cleaning or outlier removal to be performed on them (as far as we could tell).

```
# Encode discrete state from continuous observation. The -1's and min() prevent IndexError through finagling  
for i in range(len(observation)):  
    state[i] = min(np.digitize(observation[i], state_space[i])-1, q.shape[0]-1)
```

Baseline - Overall Description

```
# Turn continuous state space into discrete state space
n_bin_edges = 15
costh1 = np.linspace(env.observation_space.low[0], env.observation_space.high[0], n_bin_edges)
sinth1 = np.linspace(env.observation_space.low[1], env.observation_space.high[1], n_bin_edges)
costh2 = np.linspace(env.observation_space.low[2], env.observation_space.high[2], n_bin_edges)
sinth2 = np.linspace(env.observation_space.low[3], env.observation_space.high[3], n_bin_edges)
omega1 = np.linspace(env.observation_space.low[4], env.observation_space.high[4], n_bin_edges)
omega2 = np.linspace(env.observation_space.low[5], env.observation_space.high[5], n_bin_edges)
state_space = np.vstack((costh1, sinth1, costh2, sinth2, omega1, omega2))
```

We implemented a Q table as our baseline due to its ease of implementation, interpretability and simplicity. We imagined it to be a good benchmark to improve upon.

Each of the 6 continuous state variables discretized into 14 bins. This created a very large table and forced many physically distinct states into the same discrete category. Although the Q-table is simple and easy to implement, the discretization step fundamentally limited the model's ability to represent the fine-grained control needed for the acrobot swing-up which made learning inconsistent.

☐	 acrobot-15binedge-old.pkl	last month	172.3 MB	
☐	 acrobot-23binedge-old.pkl	last month	2.5 GB	 Too much!
☐	 acrobot.pkl	last month	172.3 MB	

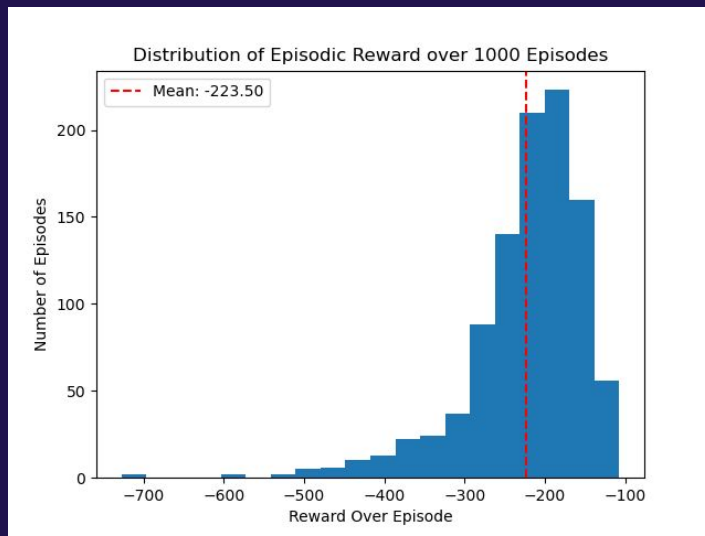
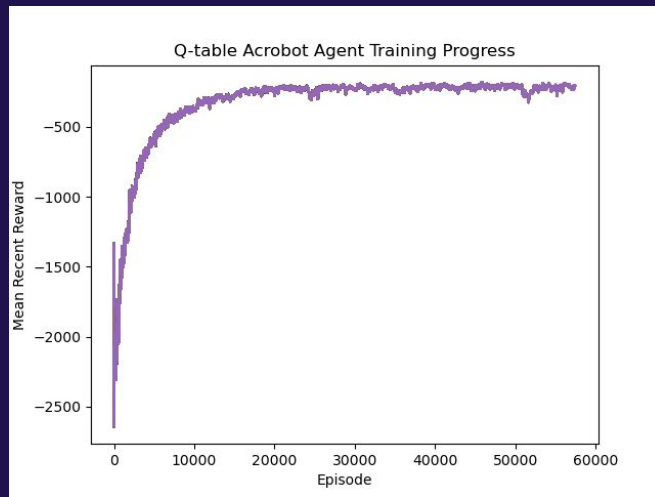
Baseline - Visualization and Results

We measured the performance of our baseline model with a plot of the mean reward over the last 100 episodes of training, and the distribution of rewards over 1000 test episodes.

The plot of training progress gave us our main metric for model success, as well as allowing us to identify convergence, and the histogram was used to verify that the results from the training plot.

During training, the Q-table quickly plateaued at a mean reward of about -225 . When evaluated over 1000 episodes, it achieved an average reward of -223.5 , far below the -100 threshold required to solve the task. Thus, the baseline model failed to solve the Acrobot problem.

Evidently, modeling a *continuous state space* with *discrete bins* was not the best choice.

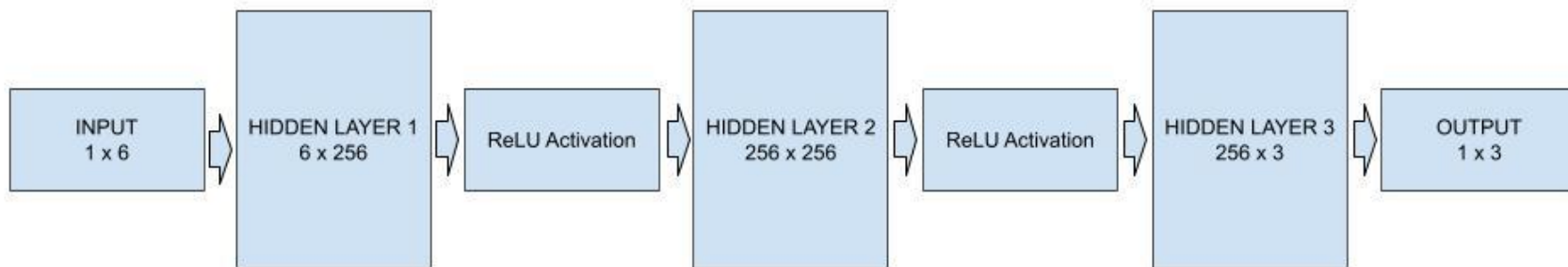


Final Model - Overall Description

To align our modeling better with the continuous nature of the state space, we changed architecture from a Q-table to a Deep Q-Network, which is capable of receiving inputs directly as floats. This means that the DQN models state more similarly to the actual problem, and that the storage requirements (and computational cost of interacting with said storage) of the model are much lower than those of the Q-table.

In training, two neural networks were created: a policy network and a target network. At each step of training, the predictions of the policy network and the target network (which uses an equation accounting more for future state) are compared, and the policy network is updated. These comparisons are made using a batch of samples from replay memory. The target network is updated with soft updates, to prevent the “moving target” problem.

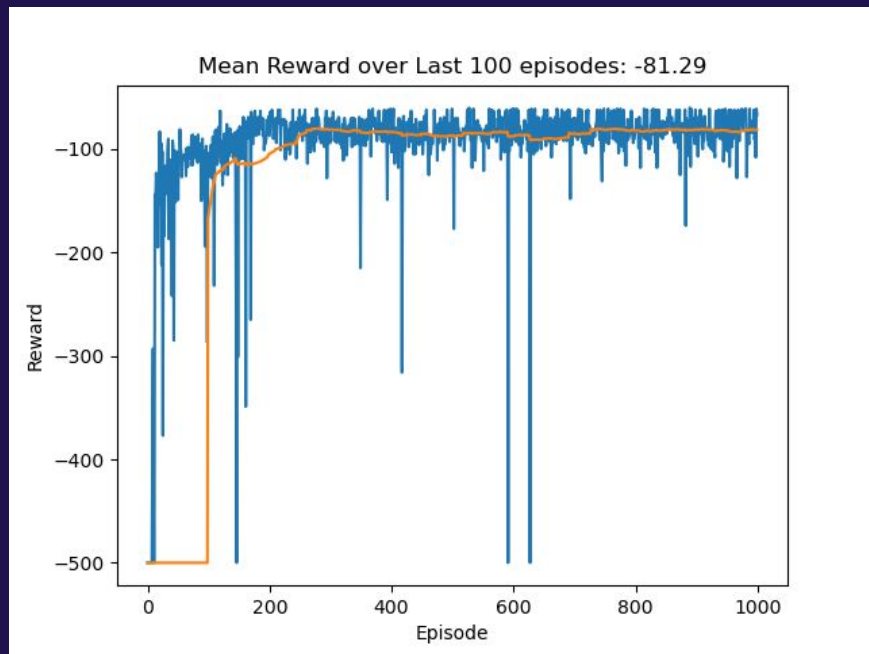
DQN Architecture:



Final Model - Visualization and Results

After 1000 training episodes, the DQN achieved a mean reward of -81.29 over the last 100 episodes. This value is greater than the reward threshold of -100 for solving the problem, so we say that this model solved the Acrobot problem.

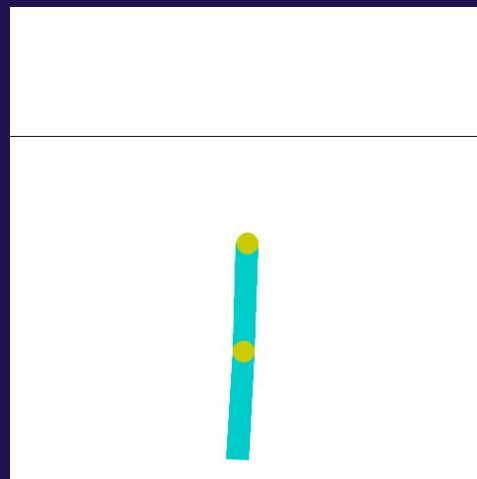
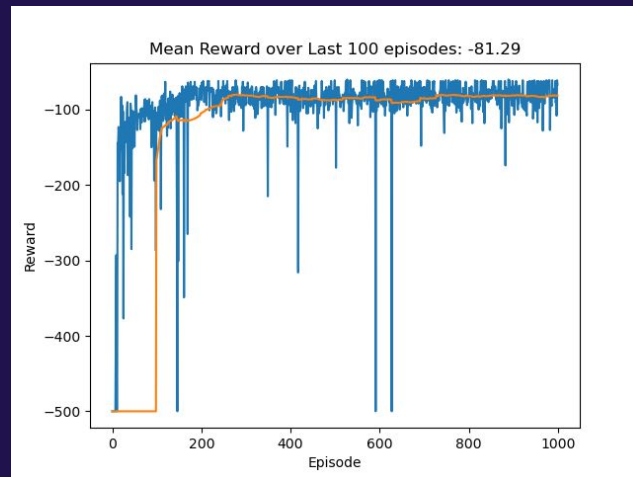
Evidently the switch to an architecture representing the continuous state space through interpolation instead of discretization was a success.



Insights on Visualization

After establishing a basic understanding of the problem, we found the plots of mean reward over the last 100 episodes to be the most useful out of the visualizations we generated, because they gave us a good metric for measuring the success of a model.

However, actually looking at the acrobat's movement through different episodes was probably the most useful in helping us to understand the problem, because of its direct physical interpretation.



Insights Overall

Working through both models gave us a clearer understanding of the acrobat's dynamics. The system is extremely sensitive to small changes in angle and velocity, which makes discretization a poor fit. We also learned that sparse rewards require long training horizons and that moving averages are essential for interpreting progress.

More generally, we learned that a careful consideration of the nature of a problem allows one to identify solutions that are more appropriate for the problem. This should theoretically be an obvious statement, but it is important to remember.

To future students approaching similar RL problems, we suggest that they write out the elements of and interactions between the observation space, action space, and reward(s), to create a good baseline understanding of the nature of system at hand before trying to make a model to interact with it

Future Directions

With more time we would have tried different DQN configurations (#nodes, #layers, epsilon decay, learning rate, activation functions, optimizers, double DQN, etc.), different NN-based solution methods (such as Actor-Critic or Proximal Policy Optimization), as well as a purely (or just more) analytical solution, as the problem seems simple enough that it could be feasible.

Final Comparison

Q-table:

- Mean reward: -223.5 (Failed the task)

DQN:

- Mean reward: -81.29 (Completed the task)

This is 14th on [this online leaderboard](#).

User	Best 100-episode performance	Write-up	Video
mallochio	-42.37 $\pm$ 4.83		taken down
marunowska	-59.31 $\pm$ 1.23		
MontrealAI	-60.82 $\pm$ 0.06		
BS Haney	-61.8	Write-up	YouTube
Felix Nica	-63.13 $\pm$ 2.65	Write-up	YouTube
Nick Kaparinos	-64.30 $\pm$ 4.10	Write-up	gif
Daniel Barbosa	-67.18	writeup	
Mahmood Khordoo	-68.63	writeup	gif
lirlli	-72.09 $\pm$ 1.15		
Tiger37	-74.49 $\pm$ 10.87	writeup	
tsdaemon	-77.87 $\pm$ 1.54		
a7b23	-80.68 $\pm$ 1.18		
Tzoof Avny Brosh	-80.73	writeup	
DaveLeongSingapore	-84.02 $\pm$ 1.46		

Us

